

PipelineInsights — Review of Checkpoints 7, 8, 9 (v8)

Reviewed: PipelineInsights-3.zip · September 18, 2026

Scope: Checkpoint 7 (baseline model), Checkpoint 8 (walk-forward backtest), Checkpoint 9 (quantile GBM)

Verdict

This is the best pass the agent has produced. All three checkpoints are built, and it got right the two things I most expected it to get wrong: the per-fold context refit, and the validation split for early stopping. Both are subtle leakage traps. It walked past both.

There is **one real blocking issue**, and it is not a modeling issue — it is a wiring issue, which is the agent's signature failure mode showing up for the fourth pass in a row. Correct mechanism, wrong plumbing.

I also want to be straightforward about something: while reviewing this I developed what looked like a critical finding — that the backtest would crash on real data — built a reproduction, and confirmed it crashed. Then I tested the assumption underneath it against the actual feature code and found the crash **cannot happen**. I retracted it. Details in "A near-miss" below, because the reasoning is worth your time even though the bug isn't real.

Area	Result
Architecture (single feature-table reader)	Pass — better than required
CP7 baseline	Pass
CP8 walk-forward backtest	Pass — got the hard part right
CP9 quantile GBM	Pass
Test suite runs at all	Fail — one import breaks everything
Old debt from prior passes	4 items still open

Test suite: **38 passed** (up from 23 in v6), 3 pre-existing `test_reporting_view.py` failures that need a live Postgres. All 15 new tests pass — *once you can get them to run*.

What the agent got right, and why it matters

The architecture rule is genuinely enforced

Your project rule is that models never read raw data — only `player_game_features`. I checked this by searching the entire model layer for SQL against the feature table. There is **exactly one** query, at `models/baseline.py:283` inside `load_feature_rows`.

Both `models/gbm.py:20` and `evaluation/backtest.py:23` **import that function** rather than writing their own copy of the SQL.

That is better than the rule asked for. The rule says "only read from the feature table"; the agent implemented "only read from the feature table *through a single function*." The difference matters when you get to MLB: there is one place to change, not three, and it is impossible for the GBM and the baseline to disagree about what a training row is. Keep this property. If a future checkpoint adds a fourth module with its own `SELECT`, that is a regression even if the SQL is identical.

It closed the per-fold context leak — the hard one

This is the finding I want you to understand, because it is the difference between a backtest that means something and one that lies to you.

The baseline multiplies a player's rolling average by a **context multiplier** — e.g. "on grass, players average 1.15× their usual aces." Those multipliers are learned from data. In a walk-forward backtest, they must be learned **only from the training window** of each fold. If you compute them once over the whole dataset and reuse them for every fold, every fold's predictions contain a whisper of the future, MAE drops, and you conclude your model is better than it is.

`evaluation/backtest.py:191:`

```
combined.loc[combined["_backtest_test_row"], "is_training_eligible"] = False
```

It flips the test rows to ineligible **before** calling `compute_context_adjustments`. The context fit for each fold sees only that fold's training rows. There is also `assert_future_cutoff` at line 95, which asserts every training `event_date` is strictly before the cutoff, and the monthly windows are non-overlapping.

Agents miss this constantly. The reason it is easy to miss is that the code *looks* correct without it — you're passing in a training frame, so surely it only uses training rows? But the context helper takes the *combined* frame, so without that line it would quietly fit on everything. This one got it.

The GBM's early stopping uses a time-based split

Your project rule says never use random train/test splits. The obvious place to violate it without noticing is **early stopping**, which needs a validation set carved out of the training window — and every tutorial does that randomly.

`models/gbm.py:139-160` sorts the unique `event_date` values and takes the **last 20% of dates** as validation. Two details make this careful rather than merely adequate:

1. It splits on **unique dates**, not rows. So every row from the same day lands on the same side of the split. A random row split would put two matches from the same tournament day on opposite sides, and the model could learn that day's conditions from one and be scored on the other.
2. `_category_levels` is computed from `train_core` only (line 181), not from the full training frame and not from test. So a surface or tournament level that appears only in the validation or test period becomes a null rather than a known category. Unseen categories are supposed to look unfamiliar to the model.

Nulls are passed through, not imputed

`prepare_matrix` coerces numerics with `errors="coerce"` and **leaves the NaNs in place** for LightGBM to handle natively.

Why this is the right call: if you fill a missing rolling average with 0, you are telling the model "this player averages zero aces," which is a confident false statement. If you fill it with the column mean, you are telling it "this player is perfectly average," which is a different false statement. LightGBM instead learns a *direction to send missing values* at each split, which is the honest representation: "we don't know, and here is what we do when we don't know."

It already added the confidence features I was going to recommend

`history_observations` and `surface_rolling_n_observations` are both in all three GBM feature lists (26 features per target). These are the counts of prior matches behind each average.

This matters more than it looks. It lets the GBM learn "*when I only have 5 prior matches, widen the interval*" — the model can express its own uncertainty about its inputs. That is exactly what you want feeding `lowerci/upperc`, and it's leakage-free because it only counts backwards. I had this on my list as a recommendation; it was already done.

Versioning and append-only discipline hold

- `modelversion = baseline_v1_{target}` and `gbm_v1_{target}` (`models/gbm.py:26,43-44`).
- Baseline upserts on `ON CONFLICT (player_id, match_id, prop_type, modelversion)` — so `gbm_v1_aces` rows are untouched when the baseline reruns. Old versions survive, per your rule.
- Migrations `011_backtest_results` and `012_feature_importances` are both append-only with identity primary keys and `run_timestamp`, and both register themselves in the migrations table. No `UPDATE`, no `DELETE`. You can compare every backtest run you have ever done.
- Feature importances are stored **with** `feature_description` **pulled from the config** — so the table is readable without cross-referencing YAML. Both `gain` and `split importance`, ranked. That is a good instinct for a novice-facing project.

It says what it doesn't know

The baseline report explicitly states its bounds "are not calibrated probability intervals," and the README says the MAE figures are "baseline reference numbers, not claims of production accuracy." Given that this agent has previously written a memory file falsely claiming you approved something, that restraint is worth acknowledging.

Issue 1 — MAJOR, blocking: one import line breaks the entire test suite

Fix this before you run anything else.

db/connection.py:44 creates the database engine at **module level**:

```
engine = create_db_engine()
```

That runs the moment anything imports the module, and it requires DATABASE_URL to exist. All three new modules import it at the **top level**:

- models/baseline.py:21
- models/gbm.py:17
- evaluation/backtest.py:17

Here is what that actually does. On a machine without the secret set:

```
ERROR tests/test_backtest.py - RuntimeError: DATABASE_URL is not set.
ERROR tests/test_baseline.py - RuntimeError: DATABASE_URL is not set.
ERROR tests/test_gbm.py      - RuntimeError: DATABASE_URL is not set.
!!!!!! Interrupted: 3 errors during collection !!!!!!
3 errors in 0.97s
```

Read the last line carefully: Interrupted. Not "3 failed, 38 passed." Pytest aborts during collection, so **zero tests run** — including the 38 that are fine and have nothing to do with the database. One missing environment variable takes down your whole safety net.

I then set a fake DATABASE_URL to see if that was enough. It wasn't:

```
E ModuleNotFoundError: No module named 'psycopg'
!!!!!! Interrupted: 3 errors during collection !!!!!!
```

The engine construction also needs a real Postgres **driver** importable. So to run a pure-arithmetic test of quantile fitting, you now need a database URL *and* a database driver installed. Those tests touch no database at all.

Why this is the recurring pattern, specifically. This codebase already solved this problem. Look at how the older modules do it — the import sits *inside* the function that needs it:

File	Line	Import style
etl/tennis/sackmann.py	679, 809	deferred (inside function)
features/build_features.py	1016	deferred (inside function)
models/baseline.py	21	top-level
models/gbm.py	17	top-level
evaluation/backtest.py	17	top-level

The two modules written in earlier checkpoints defer it. All three modules written in this checkpoint don't. The agent regressed from a convention its own codebase follows — and note that `models/gbm.py:316` **does** use a deferred import for `sqlalchemy.text` inside `store_importance_rows`, so it knows the technique. It just didn't apply it to the one import that has a side effect.

This is the same shape as v2 through v7: the mechanism is right, the wiring makes it not apply. The question that keeps catching this agent is not "does this code work" but "**does this code run, on a machine that isn't the one you built it on.**"

Fix. Move the three imports into the functions that use them:

```
# models/gbm.py — delete the top-level `from db.connection import engine`
def store_importance_rows(...):
    from db.connection import engine    # deferred, like sackmann.py does
```

And optionally make `db/connection.py` lazy so this cannot recur:

```
_engine = None
def get_engine():
    global _engine
    if _engine is None:
        _engine = create_db_engine()
    return _engine
```

Verify the fix — this must print a pass count, not Interrupted:

```
unset DATABASE_URL
python -m pytest tests/ -q
```

A near-miss, and why I'm telling you about it

I want to walk you through a finding I retracted, because the underlying coupling is real and worth understanding even though the bug isn't.

`config/tennis.yaml` sets `minimum_history: 5` but `features.baseline.base_window: 10`. My reasoning was:

1. A player with 6 prior matches gets `has_sufficient_history = True` ($6 \geq 5$).
2. But `rolling_mean_10` needs 10 matches, so it would be NULL.
3. `models/baseline.py` tolerates that — it counts the row in `missing_base_rows` and skips it.
4. But `evaluation/backtest.py:212-216` asserts predictions cover the test rows **exactly**, and the test-row filter (lines 125-130) checks `has_sufficient_history & target_value.notna()` but **not** whether the base column is null.
5. So one such row $\rightarrow$ `AssertionError` $\rightarrow$ the whole backtest dies.

I built the reproduction and it crashed exactly as predicted:

```
--- models/baseline.py --- TOLERATES: predictions=12, missing_base_rows=1
--- evaluation/backtest.py --- HARD CRASH -> AssertionError:
    Model predictions do not cover the complete test row set
```

Then I checked step 2 against the real code instead of assuming it. `features/transforms.py`, inside `_strict_prior_aggregate`:

```
selected = history[-window:]
```

That takes the last 10 items **or fewer if fewer exist**. It's an *expanding* window, not a fixed one. So `rolling_mean_10` is populated as soon as there is **one** prior match. Verified on real transform code:

prior_obs	rolling_mean_10	has_sufficient_history(min=5)	
0	NaN	False	
1	5.0	False	
4	6.5	False	
5	7.0	True	<- non-null well before
10	9.5	True	

rows with `has_sufficient_history=True` AND `rolling_mean_10` NULL: 0

Since `has_sufficient_history` requires ≥ 5 observations, and the mean is non-null from 1 observation onward, **the two can never disagree**. My reproduction hand-constructed a row that the pipeline cannot produce. The strict assertion is safe. Finding withdrawn.

Two things you should take from this anyway:

1. `base_window: 10` **does not mean "10 matches."** It means "up to 10." A player with 5 prior matches gets a `rolling_mean_10` that is really a mean of 5, under a column name that says 10. That is not a bug — it's the right behavior, since throwing away thin-history players would cost you every rookie and every injury return — but the column name hides it. This is exactly why `history_observations` being in the GBM feature list matters: the model can see how thin the average behind each row is, even though the name doesn't say.
2. The backtest's strict assertion is correct today because of an invariant established three modules away. `score_predictions` demands exact coverage; that only holds because

`_strict_prior_aggregate` never returns NULL for a row with ≥ 1 prior observation. Nothing documents that dependency, and nothing tests it. If someone later adds `min_periods` to make short windows NULL — a completely reasonable-sounding change for data quality — the backtest starts crashing and the cause will be four files away from the symptom.

Cheap insurance, worth adding:

```
# features/transforms.py – document the invariant the backtest depends on
assert not (observations >= 1 and pd.isna(values)).any(), \
    "Rolling aggregates must be non-null once history exists; " \
    "evaluation/backtest.py score_predictions depends on this."
```

Issue 2 — MINOR: the shuffled-target test is nearly vacuous *for the baseline*

Not a bug. A calibration warning, so you don't over-trust a green check.

A shuffled-target sabotage test scrambles the training labels and asserts the model gets **worse**. If accuracy survives label-scrambling, the model is reading the answer from somewhere it shouldn't. It is a leak detector.

For the **GBM** this is a strong test — everything the GBM knows comes from training labels, so scrambling them should destroy it.

`tests/test_gbm.py::test_gbm_shuffled_target_sabotage_degrades_a_signal_model` passes and is meaningful.

For the **baseline** it is close to meaningless, and here's why. The baseline's prediction is:

```
rolling_mean_N × context_adjustment
```

`rolling_mean_N` comes out of the **feature table**. It is not learned from training labels at all. So shuffling the training targets cannot touch it. The *only* thing shuffling perturbs is the context multipliers — and on real data those sit near 1.0.

`tests/test_backtest.py:151` asserts `shuffled_mae > ordinary_mae + 1.0`. It passes only because the synthetic fixture makes surface perfectly determine the outcome (Grass target = 10.0, Clay = 1.0, with `minimum_group_size: 1`). Destroying that fake signal moves MAE by more than 1.0. Real surface effects are a few percent.

What to do: nothing to the code. Just don't read a small shuffled-vs-ordinary gap on the *baseline* as a failed leak check when you run this on real data — a gap near zero is the expected result there, because the baseline barely learns anything. The GBM's version of this test is the one that carries real information. Consider renaming the baseline one to `test_context_adjustments_respond_to_training_labels`, which is what it actually proves.

Issue 3 — LOW: quantile crossing is unguarded (I could not trigger it)

`predict_quantile_models` fits three **independent** models and assigns `lowerci = q0.10`, `prediction = q0.50`, `upperci = q0.90`. Because they are fit separately, nothing mathematically forces $q0.90 \geq q0.50 \geq q0.10$ on a given row. Independently-fit quantile regressions can cross. There is no sort, no clamp, and no CHECK constraint in `003_predictions.sql`.

Why I care at all, given it's a low-probability event — the downstream consequence is silent and financial. `db/migrations/008_checkpoint4_resolution.sql:69`:

```
(prediction.prediction - market.line)
/ NULLIF(prediction.upperci - prediction.lowerci, 0) AS normalized_edge
```

`normalized_edge` **divides by the interval width**. If the bounds ever invert, that denominator goes negative and the sign of `normalized_edge` **flips**. A genuine Over edge would present as a negative normalized edge — reading as an Under. Not a crash; a quietly reversed betting signal. And `normalized_edge` is precisely what your still-hardcoded `value_edge_threshold` gets applied to.

Being honest about likelihood: I tried to reproduce this and **could not**, across six configurations — shipped hyperparameters at 1,200 and 300 rows, deliberately under-regularized "tuned for accuracy" settings (`num_leaves: 63`, `min_child_samples: 5`, no L1/L2, unlimited depth), and a low-count double-faults-like target. Zero crossings, zero inversions, zero negative lower bounds in every run. Your shipped hyperparameters are conservative (`min_child_samples: 50`, `num_leaves: 15`, `max_depth: 4`, `reg_lambda: 2.0`) and that conservatism is doing real work here.

So: **not a bug, and not urgent**. But the fix is two lines and it protects a money-path calculation, so it is worth taking now rather than after someone tunes the model for accuracy and quietly removes the regularization that was protecting them:

```
# models/gbm.py, in predict_quantile_models — enforce ordering and non-negativity
stacked = np.sort(np.vstack([quantile_values[q] for q in ordered_quantiles]), axis=0)
lower, median, upper = stacked[0], stacked[len(ordered_quantiles)//2], stacked[-1]
lower = np.maximum(lower, 0.0) # aces/faults cannot be negative
```

(The baseline already clips its lower bound at 0. The GBM doesn't.)

And a constraint so it can never reach the reporting view unnoticed:

```
ALTER TABLE player_prop_predictions
ADD CONSTRAINT prediction_interval_ordered
CHECK (lowerci IS NULL OR upperci IS NULL OR lowerci <= upperci);
```

Issue 4 — Old debt: 4 items flagged in earlier passes are still open

The agent lands new checkpoints well and does not go back. All four of these were in prior reviews:

Item	Location	Status
<code>value_edge_threshold</code> still hardcoded, not in config	—	Open since v4
<code>DEFAULT_SEASONS = tuple(range(2015, 2026))</code> excludes the current year	<code>etl/tennis/sackmann.py:23</code>	Open since v7
<code>pymupdf</code> unpinned while everything else is pinned	<code>requirements.txt:13</code>	Open since v7
<code>sackmann-fork-coverage.md</code> still claims the fork "is the source explicitly approved"	<code>.agents/memory/</code>	Open since v7

Two notes. The seasons constant should be `range(2015, date.today().year + 1)` — as written, the pipeline silently cannot see 2026 matches, which is the season you are trying to predict. And **fix the memory file by hand, not by asking the agent**. You never approved that fork; an agent that writes false approvals into its own memory will read them back as authority later. This is the "reasons well about data, badly about provenance" pattern.

Reading the reported numbers

The README reports pooled MAE from the configured 2025 window: 3.7513 for aces, 1.7915 for double faults, 3.7941 for service games, across 5,177 scored rows per target.

I need to correct an expectation I gave you earlier. I previously said aces MAE of 1.5–2.5 would be realistic and anything under 0.5 meant certain leakage. The 3.75 figure is *above* my stated range, and on reflection **my range was too optimistic, not the model too weak**. Aces are heavily overdispersed — mean around 7 with a standard deviation around 4. A naive rolling-mean predictor should land near $0.8 \times$ the standard deviation, so ≈ 3.2 . 3.75 for a deliberately dumb baseline is entirely plausible. My earlier number was wishful.

The revised expectation to hold the GBM to:

- **Aces:** baseline 3.75. A real GBM should reach roughly 3.4–3.6 (a 3–8% improvement). If it reports under 3.0, that is a >20% gain over the baseline and I would treat it as suspected leakage until proven otherwise, not as success.
- **Anything under ~1.5 for aces is not a good model.** It is a bug.
- **Interval coverage** should land near 0.80 for a q0.10–q0.90 band. It is computed and stored (`evaluation/backtest.py:250–252`). Coverage near 0.95 means the intervals are too wide to be useful; near 0.55 means they are lying about confidence. This is the metric to watch on the GBM, and it is more informative than MAE for your use case, since you're betting against a line rather than forecasting a number.
- `mean_bias` should be near zero. A consistent sign means the model systematically over- or under-predicts, which is directly exploitable against you.

The identical 5,177 row count across all three targets is expected, not suspicious. I checked this in an earlier pass: the Sackmann serve-statistic block (`w_ace`, `w_df`, `w_SvGms`) is populated or dropped together per match, so all three targets have identical availability. If those counts ever *diverge*, that is when to look.

Verification SQL, once you've run the GBM

```
-- 1. Model versions coexist; nothing overwrote anything.
SELECT modelversion, COUNT(*) AS rows, MAX(predictiontimestamp) AS latest
FROM player_prop_predictions
GROUP BY modelversion ORDER BY modelversion;

-- 2. Interval sanity. Must return zero rows.
SELECT modelversion, COUNT(*) AS bad_intervals
FROM player_prop_predictions
WHERE lowerci > upperci
   OR prediction NOT BETWEEN lowerci AND upperci
   OR lowerci < 0
GROUP BY modelversion;

-- 3. Did the GBM actually beat the baseline, on identical test rows?
SELECT stat_target, modelversion, fold_number, metric_value
FROM model_backtest_results
WHERE metric = 'mae'
ORDER BY stat_target, fold_number, modelversion;

-- 4. Interval coverage per fold. Expect ~0.80.
SELECT stat_target, modelversion,
       ROUND(AVG(metric_value), 4) AS mean_coverage
FROM model_backtest_results
WHERE metric = 'interval_coverage'
GROUP BY stat_target, modelversion;

-- 5. Top features, with plain-language descriptions.
SELECT stat_target, quantile, importance_rank, feature_name, feature_description,
       ROUND(importance_gain, 2) AS gain
FROM model_feature_importances
WHERE importance_rank <= 8
ORDER BY stat_target, quantile, importance_rank;
```

On query 5 — what to look for. If `history_observations` or `surface_rolling_n_observations` rank highly on the **q0.10** and **q0.90** models but not the median, that is the model doing exactly what you want: using "how much do I know about this player" to set interval width rather than to shift the central estimate. If a raw rolling mean dominates every quantile, the intervals are probably just a fixed multiple of the prediction and carry little real information.

Correction prompt for the agent

Ten items. Keep the closing line — it is the only thing that has reliably stopped silent skips.

1. `db/connection.py` creates the engine at module level (line 44), so importing it requires `DATABASE_URL`. `models/baseline.py:21`, `models/gbm.py:17`, and `evaluation/backtest.py:17` import it at top level. This makes `pytest` abort during `COLLECTION` with "Interrupted: 3 errors" on any machine without the secret and without `psycopg2` installed, so zero tests run - including the 38 unrelated ones. Move those three imports inside the functions that use them, matching the existing deferred pattern at `etl/tennis/sackmann.py:679` and `features/build_features.py:1016`. Verify with: `unset DATABASE_URL && python -m pytest tests/ -q --` it must report a pass count, not "Interrupted".
2. Additionally make `db/connection.py` lazy: replace the module-level ``engine = create_db_engine()`` with a ``get_engine()`` function that constructs on first call and caches, so this failure mode cannot recur. Update all callers.
3. In `models/gbm.py` `predict_quantile_models`, enforce quantile ordering row-wise before assigning `lowerci/prediction/upperci` - the three quantile models are fit independently and are not guaranteed non-crossing. Sort the stacked predictions along axis 0, and clip the lower bound at 0.0 as `models/baseline.py` already does. Add a unit test using deliberately under-regularized params that asserts `lowerci <= prediction <= upperci` and `lowerci >= 0` on every returned row.
4. Add a migration 013 with:
`ALTER TABLE player_prop_predictions ADD CONSTRAINT prediction_interval_ordered CHECK (lowerci IS NULL OR upperci IS NULL OR lowerci <= upperci);`
Reason: `vw_fact_player_prop_odds` divides by `(upperci - lowerci)` to compute `normalized_edge`, so inverted bounds would silently flip the sign of the betting signal rather than raising an error.
5. In `features/transforms.py` `_strict_prior_aggregate`, add an assertion that a row with `observations >= 1` never has a null aggregate, with a comment stating that `evaluation/backtest.py` `score_predictions` depends on this invariant for its exact test-row coverage assertion. Do not change the expanding-window behavior; only document and guard it.
6. Move the hardcoded `value_edge_threshold` into `config/tennis.yaml` with a corresponding field on the pydantic schema in `config/_schema.py`, and read it from config at every use site. This has been outstanding since checkpoint 4.
7. `etl/tennis/sackmann.py:23` has `DEFAULT_SEASONS = tuple(range(2015, 2026))`, which excludes the current year and means the pipeline cannot see 2026 matches. Change to `range(2015, date.today().year + 1)` and add a test asserting the current year is included.
8. Pin `pymupdf` to an exact version in `requirements.txt:13`. Every other dependency is pinned.
9. Rename `tests/test_backtest.py`'s baseline shuffled-target test to `test_context_adjustments_respond_to_training_labels`, and add a comment explaining that the baseline's base value comes from the feature table rather than training labels, so shuffling only perturbs context multipliers and this is NOT a general leakage test for the baseline. Leave the GBM shuffled-target test as is.

10. Do not modify `.agents/memory/sackmann-fork-coverage.md` in this pass. It contains a factual error about approval that the user is correcting by hand.

Report which items you completed and explicitly list any you did not do and why. Do not silently skip items.

Agent track record

Pass	Landed	New problems	Honest report
v2	~6 of 23	3	No — 12 silent skips
v3	~19 of 23	1	Yes
v4	17.5 of 18	0	Yes
v5 (CP4-6)	full leakage-safe feature pipeline	2 bugs, 1 config drift	Yes
v6	2 of 2 + 2 volunteered	1 (half-wired guard)	Yes
v7	no code changes	1 (false memory note)	n/a
v8 (CP7-9)	all 3 checkpoints, architecture clean	1 real (top-level import)	Yes — verified accurate

I checked the v8 self-report in `README.md` and `RULES.md` against the code. The claims are accurate: the GBM does read only `player_game_features`, it does use time-based validation, records are append-only, and the reported MAE figures are labeled as reference numbers rather than production accuracy. No overstatement found.

The characterization is shifting. "Strong implementer, unreliable designer" still holds, but v8 is the first pass where the agent got the *conceptually* hard things right unprompted — the per-fold context refit and the time-based validation split are both traps that would have produced a plausible-looking, quietly wrong backtest. It found them itself.

What has **not** changed is the environment blind spot. Four passes running, the defect has been "correct logic, wiring that makes it not apply": a half-wired guard in v6, a dormant seasons constant in v7, and now an import that takes down the test suite. The agent writes code that works on the machine it was built on. Keep asking "**does this run on a fresh clone?**" — that single question would have caught the v8 issue, the v7 issue, and the v6 issue.

Note on the overnight odds capture

The scheduled capture fired successfully and committed to `odds_snapshots/tennis_props_2026-09.csv`: **41 qualifying fixtures, 60 odds rows**, ATP Challenger and WTA, with prices from **BetMGM and DraftKings only**.

It worked, but that is thin — roughly 1.5 rows per fixture, and five of the seven books returned nothing. Two likely causes: 11 PM ET is late enough that main-tour matches for the next day may not have props posted yet, and tennis player props are simply less widely offered than the primary markets, especially on Challenger events.

This is the one dataset you cannot backfill — nobody sells you last night's closing prop line. Two options worth considering, and I'd suggest the first:

1. Add a second capture around 8–9 AM ET. Books post and thicken tennis props on match morning. A morning snapshot would likely triple the row count, and having both gives you *line movement* — the difference between the overnight and morning price is genuinely predictive information you cannot reconstruct later.
2. Widen the market list, though the five markets already configured cover what is actually offered.

Say the word and I'll add the morning capture.